

Concurrency

Java Advanced Developer

StayAhead Training; Oct, 2025

What is Concurrency?

Concurrency

- A. Doing two or more things simultaneously, (AKA parallelism) or...
- B. Switching between tasks so quickly to give the impression of A



Imagine Superman using his super speed to scan items at every aisle in your local supermarket

Threads

Concurrency

- A lightweight process/thread of execution
- On startup the VM is comprised of one thread that executes `main()`
- New threads may be spawned to exploit concurrency (multithreading)
- **No guarantees** about when a new thread starts or order of execution

Spawning a Thread: Thread Subclass

Concurrency

```
class MyThread extends Thread {  
  
    @Override  
    public void run() {  
        // this code will be executed by a new thread  
        System.out.println("Hello world");  
    }  
}  
  
...  
var myThread = new MyThread();  
myThread.start();
```

DON'T call run!

Spawning a Thread: Runnable Impl.

Concurrency

```
class MyRunnable implements Runnable {  
  
    @Override  
    public void run() {  
        // this code will be executed by a new thread  
        System.out.println("Hello world");  
    }  
}  
  
...  
var myThread = new Thread(new MyRunnable());  
myThread.start();
```

Spawning a Thread: Runnable Lambda

Concurrency

- Note that Runnable is a functional interface

```
var myThread = new Thread(  
    () -> System.out.println("Hello world"));  
  
myThread.start();
```


Identifying Threads

Concurrency

```
var thread1 = new Thread(() -> {  
    Thread thisThread = Thread.currentThread();  
    System.out.println(thisThread.getName());  
});
```

```
thread1.start();  
Thread mainThread = Thread.currentThread();  
System.out.println(mainThread.getName());
```

// the name of which thread will be output first?

Pausing a Thread

Concurrency

```
var thread1 = new Thread() -> {  
    try {  
        Thread.sleep(1000);  
    } catch (InterruptedException e) {  
        // TODO handle interrupt?  
    }  
    Thread thisThread = Thread.currentThread();  
    System.out.println(thisThread.getName());  
});  
  
thread1.start();
```


Waiting for a Thread to Finish

Concurrency

```
var thread1 = new Thread() -> {  
    IntStream.range(1, 11).forEach(System.out::println);  
});
```

```
thread1.start();  
thread1.join();  
System.out.println("Thread #1 is finished");
```

Force the main thread to wait for thread1 to finish

Stopping a Thread

Concurrency

- Thread class's ~~stop~~ method is deprecated; use a **boolean** field, e.g.:

```
@Override
public void run() {
    while (true) {
        if (finished) break;
        // TODO the thread's work
    }
}
...
thread.finished = true;
```

Stopping a Thread

Concurrency

- Alternatively interrupt the thread by calling its `interrupt` method, e.g.:

```
@Override
public void run() {
    while (true) {
        if (isInterrupted()) break;
        // TODO the thread's work
    }
}
...
thread.interrupt();
```


Daemon Threads

Concurrency

- A thread that does not keep the VM running after the main thread finishes

```
var thread1 = new Thread(() -> {  
    System.out.println("Hello world");  
});
```

```
thread1.setDaemon(true);  
thread1.start();
```

ExecutorService

Concurrency

- An object that simplifies and makes more efficient the spawning of threads
- A pool of one or more reusable threads
- No need to code classes or call `start` or `join`
- `java.util.concurrent.ExecutorService`

ExecutorService: Types

Concurrency

Executors Factory method	ExecutorService Description
<code>newSingleThreadExecutor</code>	One thread only
<code>newFixedThreadPool</code>	N threads; for few long-running tasks
<code>newCachedThreadPool</code>	Creates threads on demand; for many short-lived tasks
<code>newScheduledThreadPool</code>	Executor that can set tasks to run after a timeout/periodically

ExecutorService: execute

Concurrency

```
ExecutorService executor =  
    Executors.newSingleThreadExecutor();
```

```
// execute accepts a Runnable; no return value  
executor.execute() -> System.out.println("Hello"));
```

```
// wait for existing tasks to finish; don't accept new ones  
// or initialise ExecutorService in try-with-resources  
executor.shutdown();
```

```
// see also shutdownNow and awaitTermination
```

Future

Concurrency

- An object that wraps the result of a thread's work
- Has methods to check if the task is complete and to wait for it be so, e.g.:

```
if (future.isDone()) {  
    // blocks the thread until the task is complete  
    var result = future.get();  
}
```

- `java.util.concurrent.Future`

ExecutorService: submit (w/Runnable)

Concurrency

```
ExecutorService executor =  
    Executors.newSingleThreadExecutor();  
  
Future future = executor.submit(() -> {  
    // TODO some long-running task  
});  
  
// wait for the task to complete  
future.get();  
  
executor.shutdown();
```

Callable

Concurrency

- `Callable` is a functional interface
- Its one method accepts nothing and returns something (it's generic)
- `ExecutorService`'s `submit` method is overloaded to accept a `Callable`
- It facilitates tasks that produce a value

ExecutorService: submit w/Callable

Concurrency

```
ExecutorService executor =  
    Executors.newSingleThreadExecutor();  
  
// this time the submit method is passed a Callable  
Future future = executor.submit(() -> "Hello");  
  
String greeting = future.get();  
// greeting ==> Hello  
  
executor.shutdown();
```


ExecutorService: invokeAny

Concurrency

```
ExecutorService executor =  
    Executors.newSingleThreadExecutor();  
  
List<Callable<Integer>> callables = List.of(  
    () -> 1, () -> 2, () -> 3, () -> 4, () -> 5);  
  
// invokeAny blocks the thread until one task completes  
// the others are then cancelled  
int num = executor.invokeAny(callables);  
  
executor.shutdown();
```

ExecutorService: invokeAll

Concurrency

```
ExecutorService executor =  
    Executors.newSingleThreadExecutor();
```

```
List<Callable<Integer>> callables = List.of(  
    () -> 1, () -> 2, () -> 3, () -> 4, () -> 5);
```

```
List<Future<Integer>> futures = executor.invokeAll(callables);
```

```
// TODO process futures
```

```
executor.shutdown();
```

Platform Threads

Concurrency

- A thin wrapper for an OS thread; captures the OS thread for its lifetime
- Platform threads impose significant overheads:
 - thousands of CPU instructions to start
 - megabytes of memory
- Platform threads are a finite resource (in terms of performance benefit)
- Blocked/waiting platform threads are a waste of resource
- Traditional solution is asynchronous programming with callbacks

Virtual Threads

Concurrency

- A thread that **is not** tied to an OS thread
- When blocked the virtual thread is swapped out for another by the VM
- The OS thread is freed up to do other things
- Good for IO-intensive tasks, **not** CPU-intensive tasks
- NB: virtual threads are daemons

Virtual Threads: from Builder

Concurrency

```
Runnable task = () -> someLongRunningTask();  
Thread.Builder threadBuilder = Thread.ofVirtual();
```

```
Thread thread1 = threadBuilder.unstarted(task);  
thread1.start();
```

```
// or...
```

```
Thread thread2 = threadBuilder.start(task);
```

```
// or...
```

```
Thread thread3 = Thread.ofVirtual().start(task);
```


Virtual Threads: from Executor Concurrency

```
Runnable task = () -> someLongRunningTask();
```

```
Executor executor =  
    Executors.newVirtualThreadPerTaskExecutor();
```

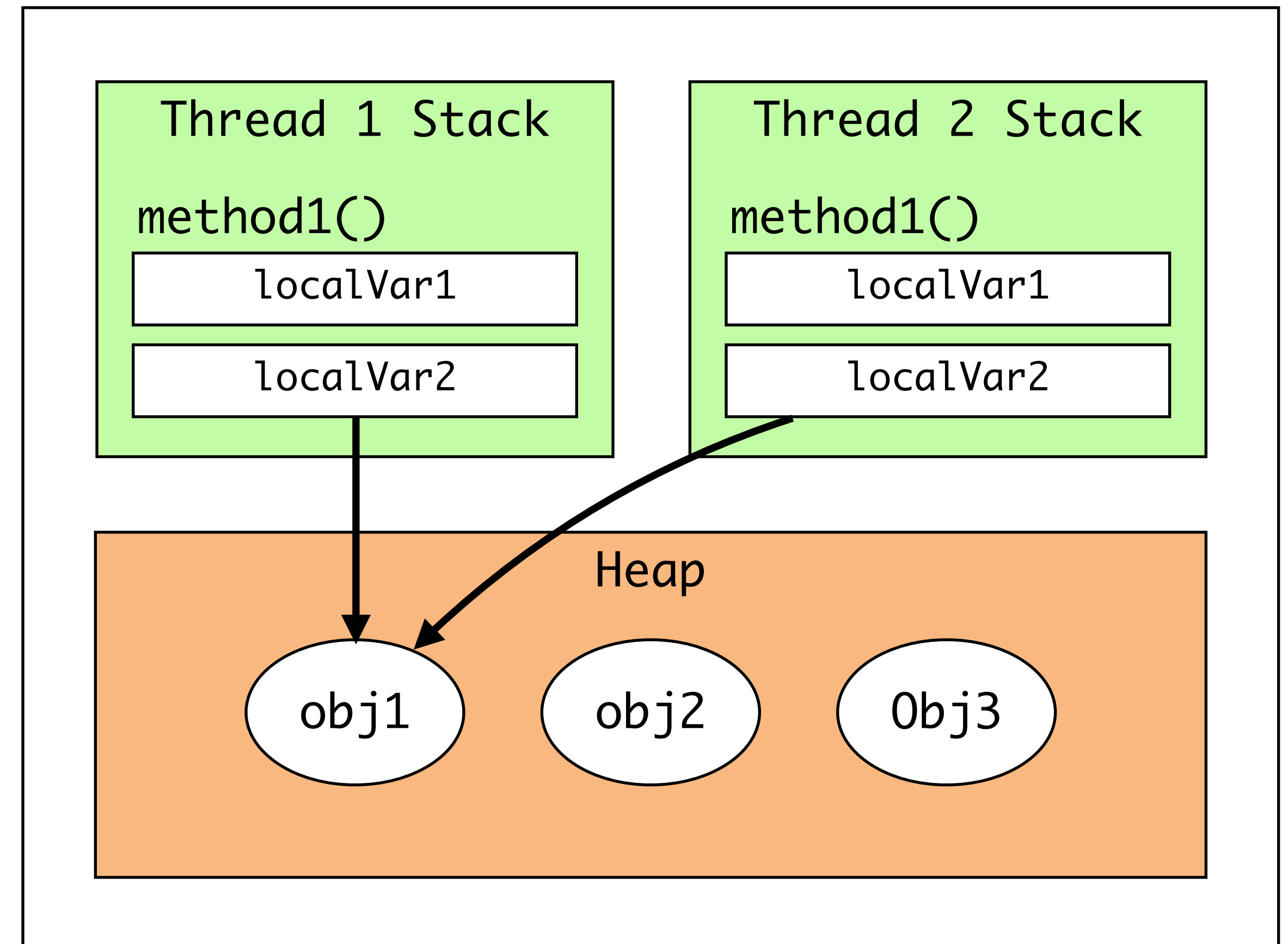
```
executor.execute(task);
```

```
executor.shutdown();
```

Java Memory Model

Concurrency

- Stack:
one per thread; contains methods in execution and their local variables
- Heap:
shared by all threads;
contains objects
- Two or more threads may share references to the same object on the heap



Data Sharing

Concurrency

```
var nums = new ArrayList<Integer>();  
var thread1 = new Thread(() -> nums.add(1));  
var thread2 = new Thread(() -> nums.add(2));
```

The ArrayList object resides on the heap and is shared by both threads

// or...

```
class MyRunnable implements Runnable {...}  
var myRunnable = new MyRunnable();  
var executor = Executors.newFixedThreadPool(10);  
executor.execute(myRunnable);  
executor.execute(myRunnable);
```

The execute method is passed the same Runnable instance each time it's called

Race Conditions

Concurrency

- Where two or more threads race to execute a critical section of code
- Where the result of executing some code by two or more threads differs depending on the order in which the threads execute the code
- Two types: read-modify-write & check-then-act
- Race conditions occur when objects are shared between threads

Race Conditions: Read-Modify-Write

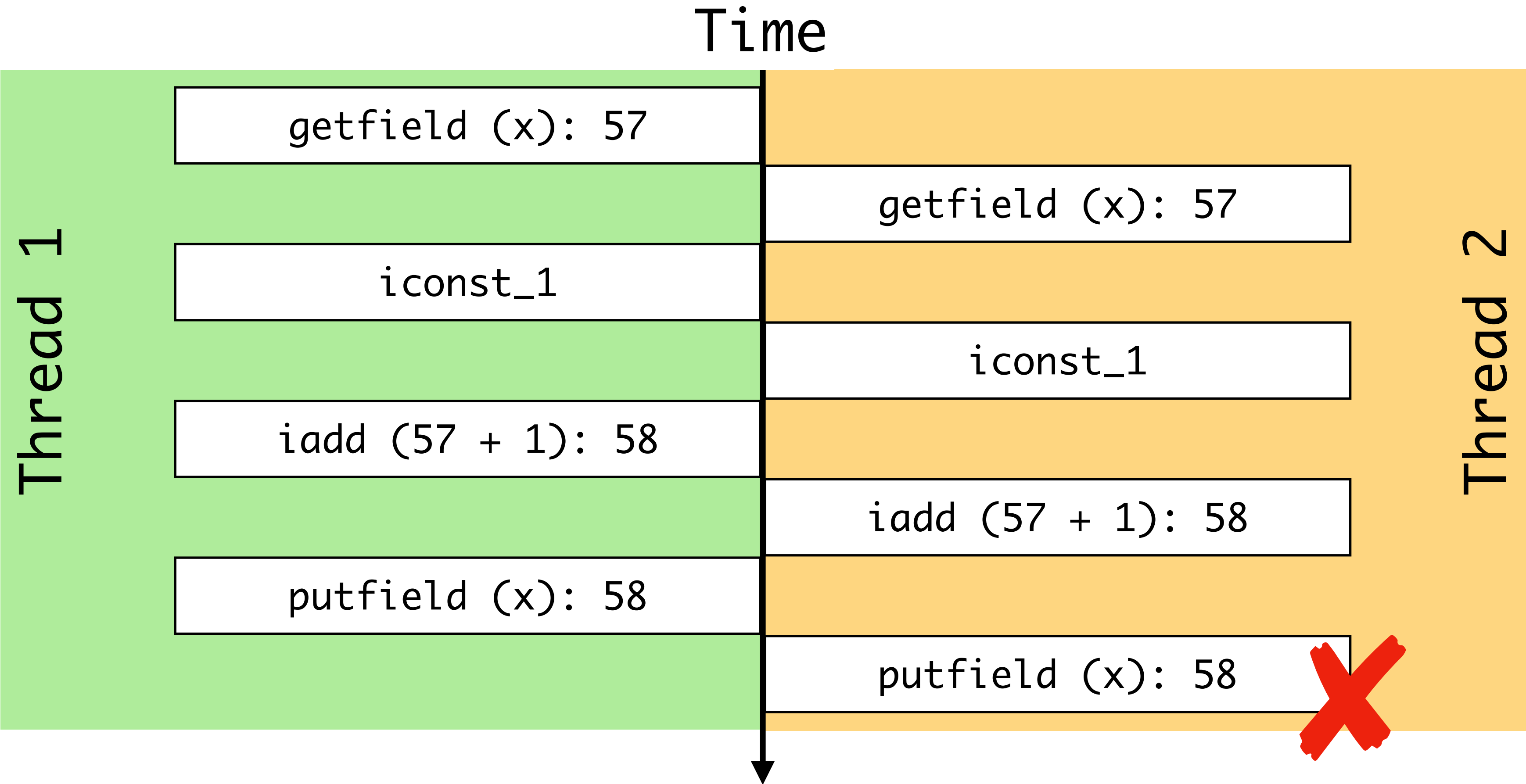
Concurrency

```
// assuming x is a field on an object shared by n threads  
x += 1
```

```
// as bytecode  
getfield      (get the field value - x)  
iconst_1      (load the int value 1)  
iadd          (add two ints)  
putfield      (set the field value - x)
```

Race Conditions: Read-Modify-Write

Concurrency



Race Conditions: Check-Then-Act

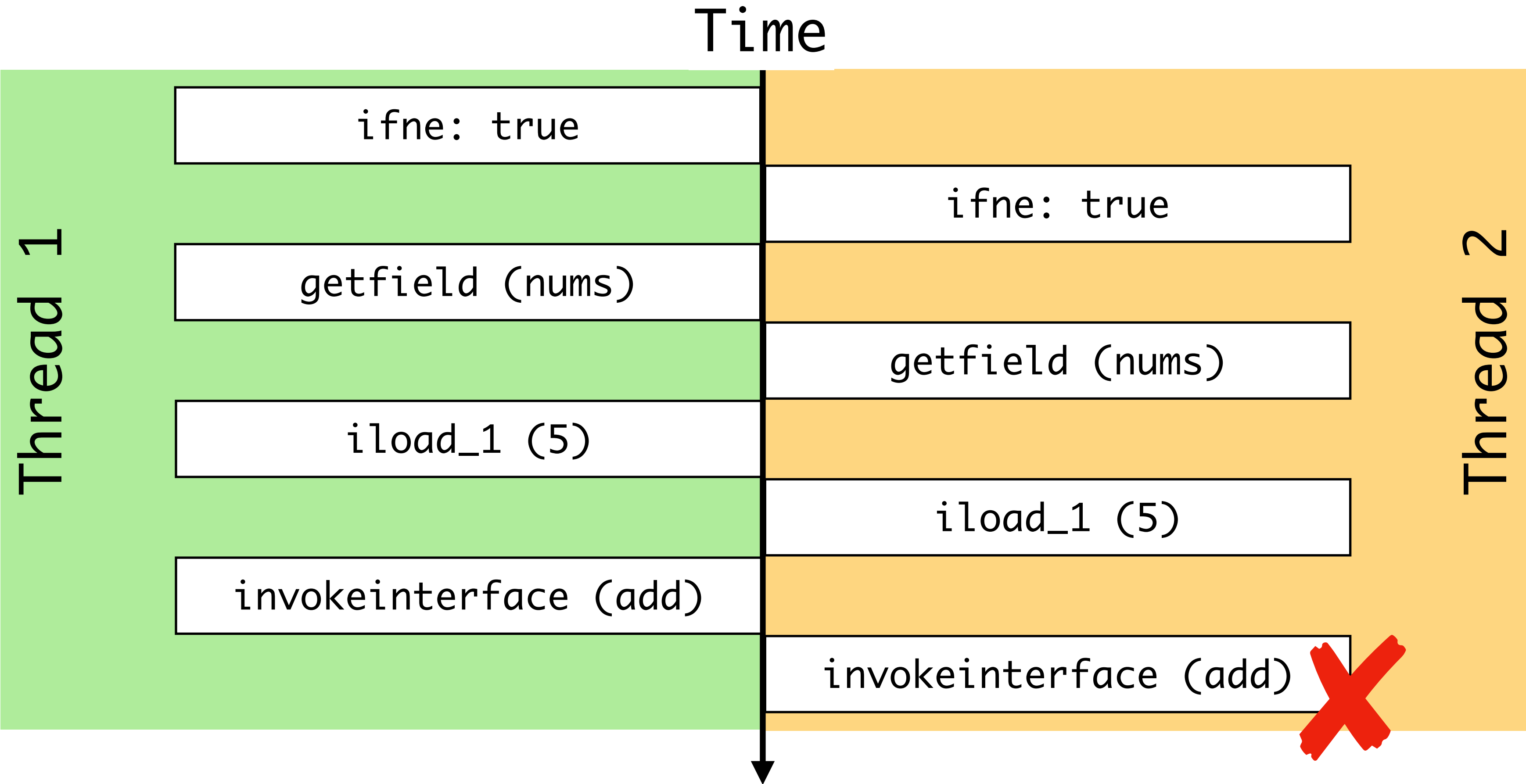
Concurrency

```
// assuming nums is a field on an object shared by n threads
if (!nums.contains(num)) {
    nums.add(num);
}
```

```
// as bytecode (subset)
invokeinterface    (call the contains method)
ifne               (if the value is zero)*
getfield           (get the field value - nums)
iload_1            (load the int value from local variable)
invokeinterface    (call the add method)
```

Race Conditions: Check-Then-Act

Concurrency



Thread Safety

Concurrency

- Code that can be executed by two or more threads safely is thread-safe
- Local primitive variables are thread-safe (the value exists only on the stack)
- Local reference variables reference objects on the heap and may be shared
- Instance fields are shared if two or more threads access **the same instance**

Immutability

Concurrency

- An immutable object is one whose fields:
 - are assigned values/immutable objects only
 - cannot be reassigned (*final*)
- Immutable objects are thread-safe, but...
 - references to them may not be
 - two or more of them may access the same external resource

Immutability

Concurrency

// is an instance of this class thread-safe?

```
class Integers {  
  
    private final int[] values;  
  
    Integers(int[] values) {  
        this.values = values;  
    }  
  
    int[] getValues() { return values; }  
}
```

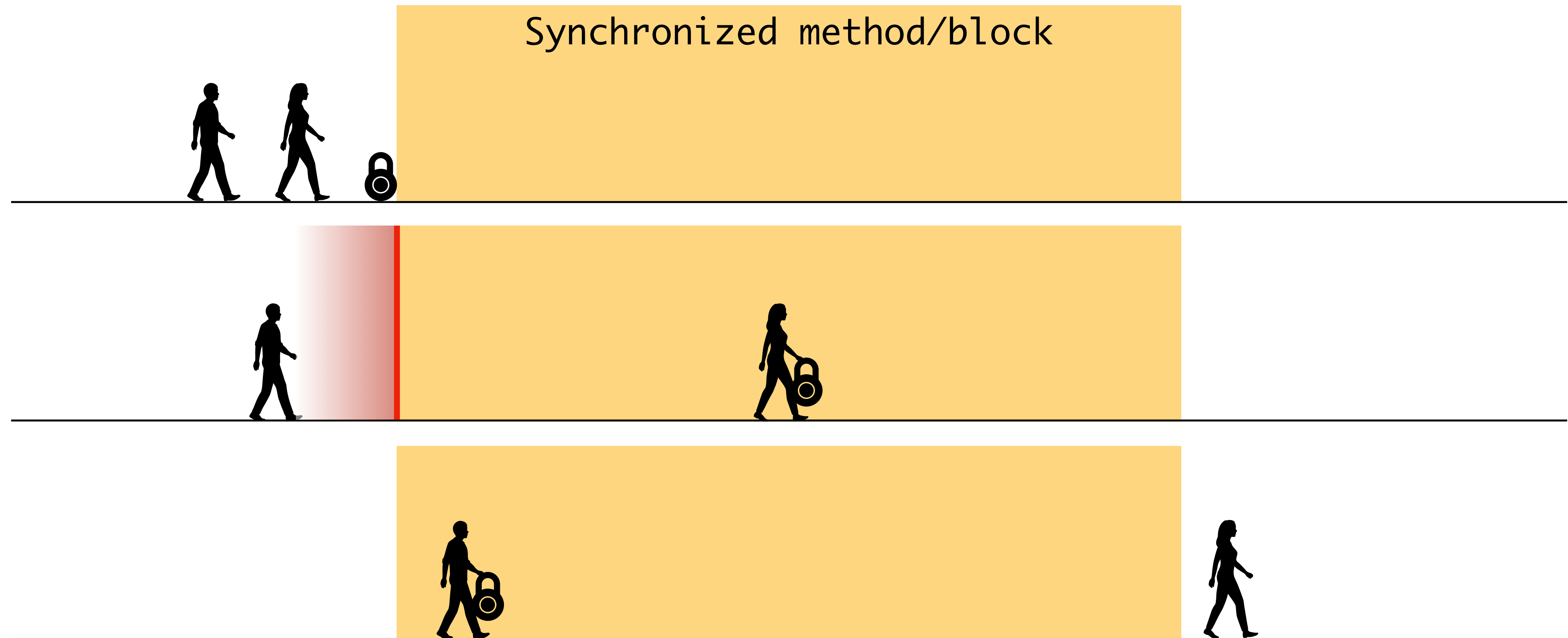

Synchronized Methods & Blocks

Concurrency

- A lock is an object that controls access to a shared resource
- Only one thread may possess the lock at any given time
- A thread must acquire the lock to enter a synchronized method/block
- This ensures only one thread may execute the critical section at a time
- A thread releases the lock when it exits the synchronized method/block

Synchronized Methods & Blocks

Concurrency



Synchronized Methods

Concurrency

```
class Counter {  
    private int value;  
  
    // the method is synchronized on the instance  
    synchronized void increment() {  
        value += 1;  
    }  
  
    int getValue() { return value; }  
}
```


Synchronized Methods

Concurrency

```
class StaticCounter {  
    private static int value;  
  
    // the method is synchronized on the class object  
    static synchronized void increment() {  
        value += 1;  
    }  
  
    static int getValue() { return value; }  
}
```

Synchronized Blocks

Concurrency

...

// only part of the method is synchronized

```
void increment() {
```

```
    System.out.println("Incrementing...");
```

```
    synchronized(this) {
```

```
        value += 1;
```

```
    }
```

```
    System.out.println("Done incrementing!");
```

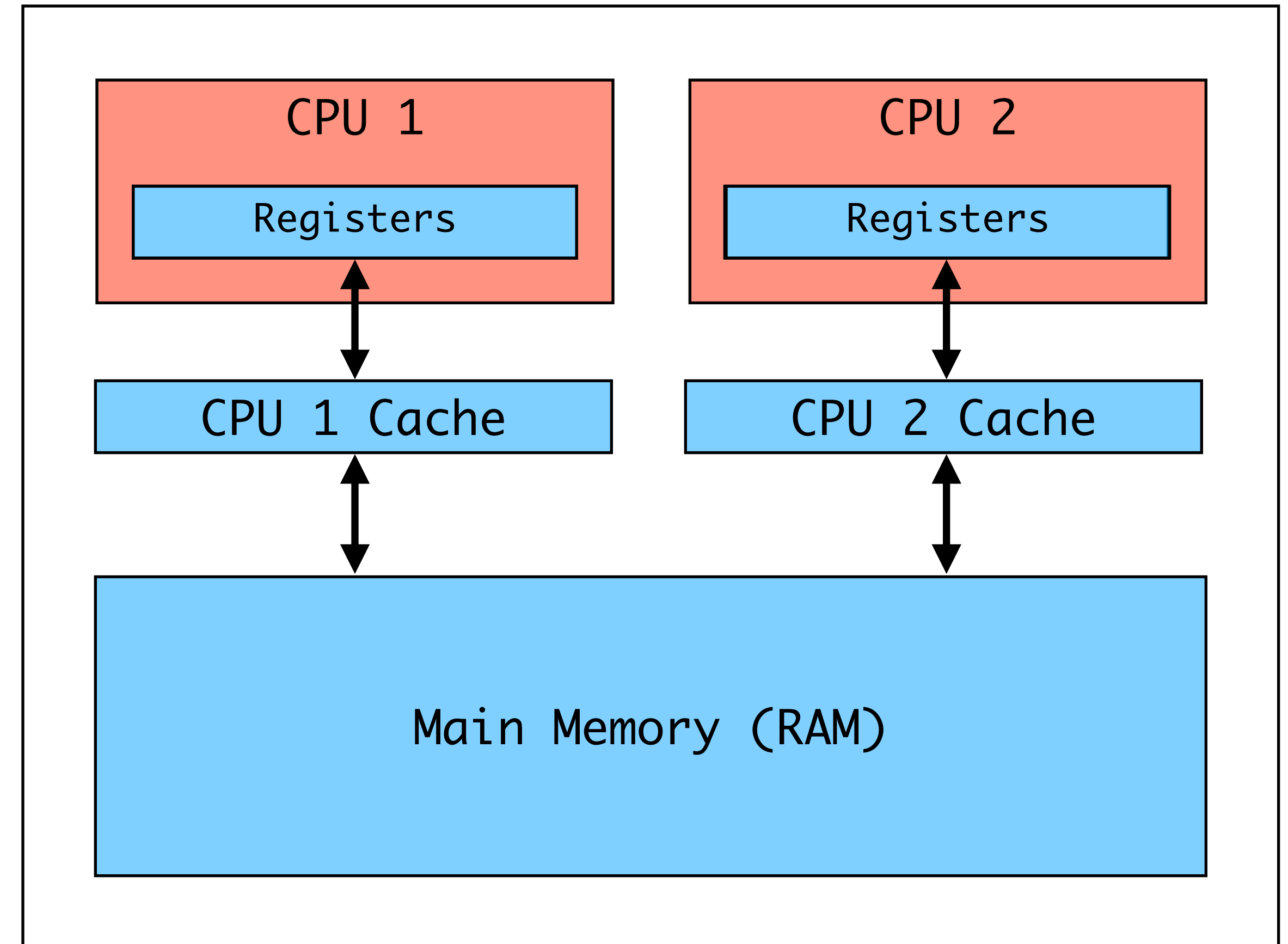
```
}
```

A reference to the lock is added in the parentheses; it need not be this, but MUST NOT be a String or primitive wrapper

Hardware Memory Model

Concurrency

- CPU registers:
very small; instant access
- CPU cache:
larger than registers;
near-instant access
- Main memory (RAM):
very large; slower than cache;
faster than disk; Java stacks and
heap reside here



Synchronized Methods & Blocks

Concurrency

- Without synchronization there are no guarantees about:
 - when changes to a shared variable will be visible to other threads
 - when a variable stored in a CPU register/cache will be committed to RAM
- Variables visible to a thread are:
 - refreshed from main memory when it enters a synchronized method/block
 - committed to main memory when it exits a synchronized method/block

Synchronized Methods & Blocks

Concurrency

- Synchronized methods & blocks:
 - permit only one thread regardless of intent
 - carry a performance overhead
 - do not synchronize across VMs
- A `ReadWriteLock` provides for sophisticated semantics (concurrent reads)
- A `Semaphore` can permit *n* threads to execute the method/block

The `volatile` Keyword

Concurrency

- `volatile` fields are always read from & written to main memory
- Visibility of the field is guaranteed between threads
- The guarantee extends to all variables visible to the thread (those other variables are read/written when the volatile field is read/written)
- The happens-before guarantee prevents potentially harmful instruction reordering
- **No good** when n threads write a new value based on the existing value (see race conditions on slides 28-30)

The `volatile` Keyword

Concurrency

```
class Counter {
```

```
    private volatile int value;
```

```
    void increment() {  
        value += 1;  
    }
```

The new value is written to main memory, not the CPU register or cache;
--- ONE THREAD ONLY! ---

```
    int getValue() { return value; }  
}
```

The existing value is read from main memory; n threads OK

The `volatile` Keyword

Concurrency

```
private volatile int a; private int b;
```

```
void incrementAll() {  
    b += 1;  
    a += 1;  
}
```

When a is written to
main memory so is b

```
void printAll() {  
    System.out.println(a);  
    System.out.println(b);  
}
```

When a is read from
main memory so is b

ThreadLocal

Concurrency

- A ThreadLocal variable can be read & written by one thread only
- Each thread gets its own ThreadLocal (like a local variable)

```
ThreadLocal<Integer> n1 = new ThreadLocal<>();  
n1.set(1);
```

```
// or withInitial which accepts a Supplier  
ThreadLocal<Integer> n1 = ThreadLocal.withInitial(() -> 1);  
  
System.out.println(n1.get());
```

ThreadLocal

Concurrency

```
class MyRunnable implements Runnable {
```

```
    ThreadLocal<Integer> num = new ThreadLocal<>();
```

```
    ...
```

```
}
```

```
var myRunnable = new MyRunnable();
```

```
var executor = Executors.newFixedThreadPool(4);
```

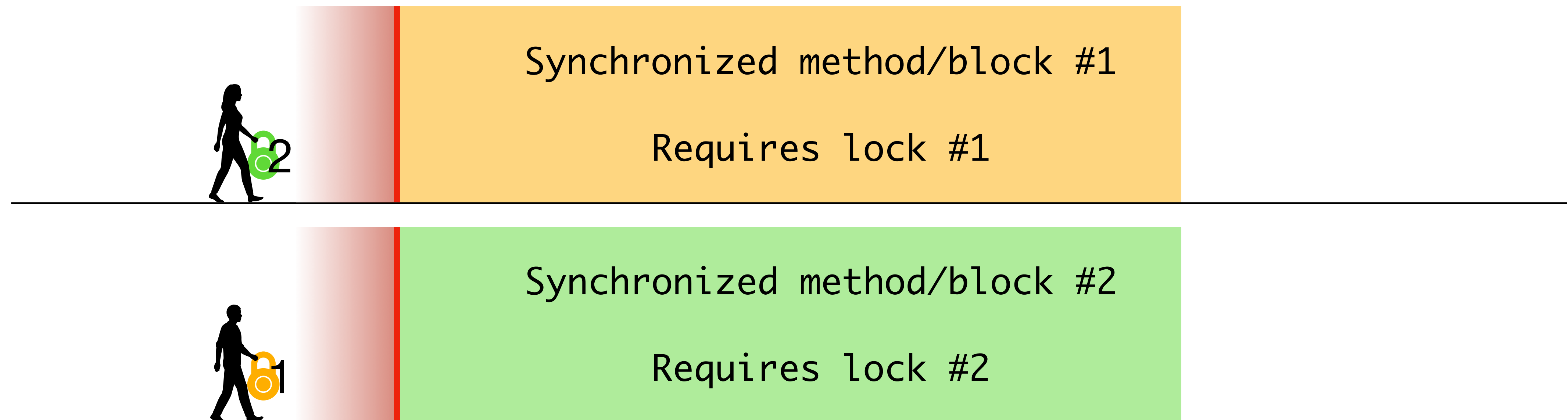
```
executor.execute(myRunnable);
```

```
executor.execute(myRunnable);
```

No shared data

Deadlock

Concurrency



Deadlock

Concurrency

// in the Movie class

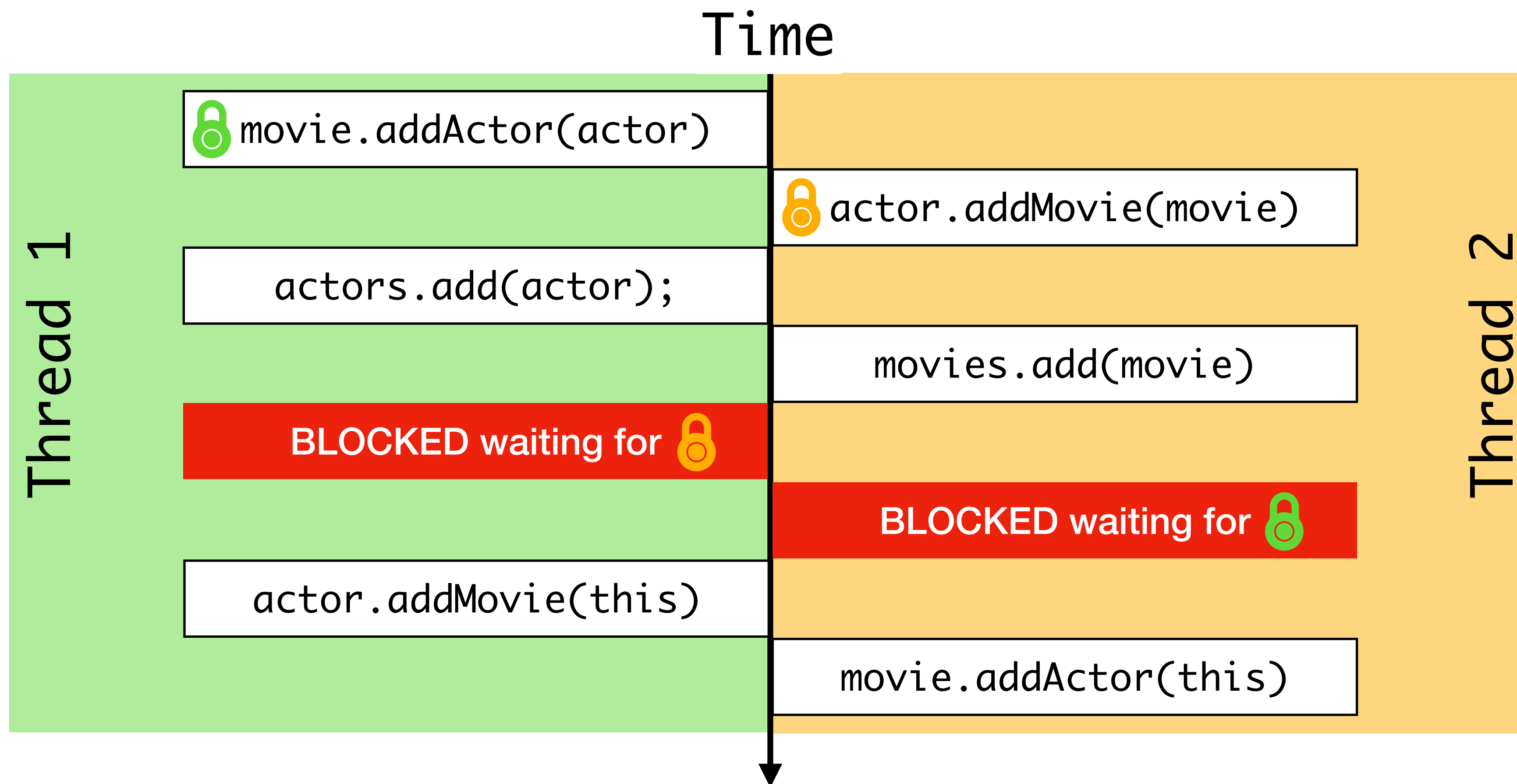
```
synchronized void addActor(Actor actor) {  
    actors.add(actor);  
    actor.addMovie(this);  
}
```

// in the Actor class

```
synchronized void addMovie(Movie movie) {  
    movies.add(movie);  
    movie.addActor(this);  
}
```

Deadlock

Concurrency



Synchronized Collections

Concurrency

```
var nums = new ArrayList<Integer>();  
nums.addAll(1, 2, 3, 4, 5);  
synchronizedNums = Collections.synchronizedList(nums);  
  
// MUST manually synchronize when traversing (incl. Streams)  
synchronized(synchronizedNums) {  
    for (var num : synchronizedNums) {  
        // TODO  
    }  
}  
// see also synchronizedMap and synchronizedSet
```

Concurrent Collections

Concurrency

- Collection implementations in the `java.util.concurrent` package
- **Not** like synchronized collections
- Thread-safe but not governed by a single exclusion lock
- May permit concurrent reads and writes
- No `ConcurrentModificationExceptions`; Iterators not to be shared

CopyOnWriteArrayList

Concurrency

- Thread-safe variant of `ArrayList`
- All mutating operations implemented by copying the underlying array
- Snapshot-style iterator means no synchronization is needed for traversal
- The Iterator will **not** reflect changes since it was created
- For when traversals vastly outnumber mutations
- See also `CopyOnWriteArraySet`

ConcurrentLinkedQueue

Concurrency

- Thread-safe implementation of Queue
- Unbounded and non-blocking
- Weakly consistent iterator means no synchronization is needed for traversal
- The iterator will reflect the queue state at some point since the iterator's creation
- See also ConcurrentLinkedDeque and various blocking variants

ConcurrentHashMap

Concurrency

- Thread-safe implementation of Map (more like Hashtable than HashMap)
- Retrieval operations do not block and may overlap with mutations
- Retrievals reflect the result of the most recent mutation
- The iterator will reflect the map state at some point since the iterator's creation
- Specify an initial capacity where possible to avoid slow resizing

ConcurrentHashMap: Slipped Condition

Concurrency

```
var map = new ConcurrentHashMap<String, Integer>();
```

```
...
```

```
if (!map.containsKey(key)) {  
    map.put(key, value);  
}
```

While each method is thread-safe, the if statement as a whole is not

```
// do instead...
```

```
map.putIfAbsent(key, value);
```



Atomic Objects

Concurrency

- Objects that can be read and written atomically (an all or nothing operation)
- All have get & set methods that simulate reads & writes to `volatile` variables
- Also `getAndSet` (swap) and `compareAndSet` (set if value equals)
- Boolean, Integer, Long, Array, and Reference variants
- Integer and Long variants have methods for incrementing and decrementing

Atomic Objects

Concurrency

```
class Counter {  
    private AtomicInteger value = new AtomicInteger(0);  
  
    void increment() {  
        value.incrementAndGet();  
    }  
  
    int getValue() { return value.get(); }  
}
```

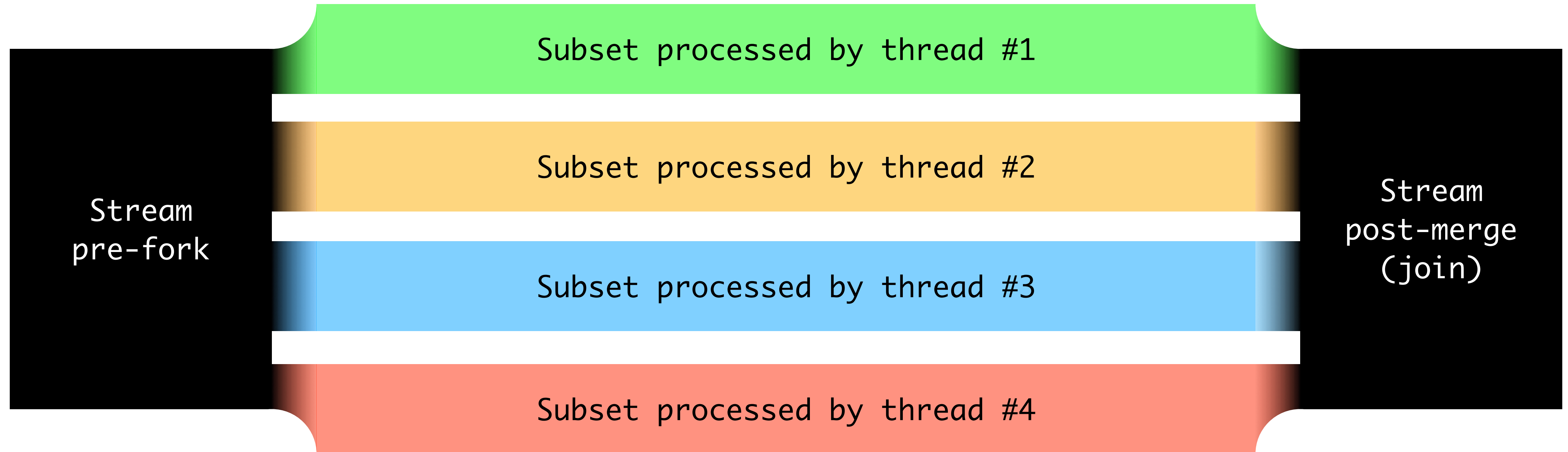

Parallel Streams

Concurrency

- A `java.util.stream.Stream` may be processed in parallel
- The work is recursively subdivided and each part processed concurrently
- Partial results are merged after processing
- Implemented using Java's Fork/Join framework
- The # of threads in the Fork/Join pool is aligned with the # of CPU cores

Parallel Streams

Concurrency



Parallel Streams

Concurrency

```
var nums = new ArrayList<Integer>();
```

```
...
```

```
Stream<Integer> parallelStream = nums.stream().parallel();
```

```
// or...
```

```
Stream<Integer> parallelStream = nums.parallelStream();
```


Parallel Streams: Considerations

Concurrency

- Splitting the stream can be expensive; an array of primitives has excellent data locality; a LinkedList of references has poor data locality
- Some Streams are hard to split, e.g. `Stream.generate(...)`
- Avoid accessing external mutable objects and intermediate operations that require shared state, e.g. `limit`, `skip`, and `findFirst`
- "Premature optimisation is the root of all evil" - Donald Knuth
- *Data locality refers to the physical proximity of the data to the processor